

算法分析

姓名：李盛鹏 学号：2351136 日期：2025年5月30日

完全平方数

代码实现

```
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, INT_MAX);
        dp[0] = 0;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j * j <= i; j++) {
                dp[i] = min(dp[i], dp[i - j * j] + 1);
            }
        }
        return dp[n];
    }
};
```

通过 589 / 589 个通过的测试用例

q5aU0co72r 提交于 2025.05.30 16:23

🕒 执行用时分布

43 ms | 击败 81.89%

📈 复杂度分析

💾 消耗内存分布

12.97 MB | 击败 49.90%

```
1 class Solution {
2 public:
3     int numSquares(int n) {
4         vector<int> dp(n + 1, INT_MAX);
5         dp[0] = 0;
6         for (int i = 1; i <= n; i++) {
7             for (int j = 1; j * j <= i; j++) {
8                 dp[i] = min(dp[i], dp[i - j * j] + 1);
9             }
10        }
11        return dp[n];
12    }
13};
```

水壶问题

代码实现

```
class Solution {
public:
    bool canMeasureWater(int x, int y, int z) {
        using pii = pair<int, int>;
        stack<pii> stk;
        stk.emplace(0, 0);
        auto hash_function = [](const pii& o) {return hash<int>()(o.first) ^
hash<int>()(o.second);};
        unordered_set<pii, decltype(hash_function)> vis(0, hash_function);
        while (stk.size()) {
```

```
        auto st = stk.top();
        stk.pop();
        if (vis.count(st)) {
            continue;
        }
        vis.emplace(st);
        auto [i, j] = st;
        if (i == z || j == z || i + j == z) {
            return true;
        }
        stk.emplace(x, j);
        stk.emplace(i, y);
        stk.emplace(0, j);
        stk.emplace(i, 0);
        int a = min(i, y - j);
        int b = min(j, x - i);
        stk.emplace(i - a, j + a);
        stk.emplace(i + b, j - b);
    }
    return false;
};
```

题目描述 | 题解 | 通过 | 提交记录

全部提交记录

通过 32 / 32 个通过的测试用例

发门无敌了 提交于 2025.05.30 16:29

官方题解 写题解

百度面试突击手册

众里寻你签百度

执行用时分布

0 ms | 击败 100.00%

复杂度分析

消耗内存分布

8.97 MB | 击败 27.06%

代码

C++ 智能模式

```
3 bool canMeasureWater(int x, int y, int z) {
4     using pii = pair<int, int>;
5     stack<pii> stk;
6     stk.emplace(0, 0);
7     auto hash_function = [](const pii& o) {return hash<int>()(o.first) ^ hash<int>()(o.
8 second);};
9     unordered_set<pii, decltype(hash_function)> vis(0, hash_function);
10    while (stk.size()) {
11        auto st = stk.top();
12        stk.pop();
13        if (vis.count(st)) {
14            continue;
15        }
16        vis.emplace(st);
17        auto [i, j] = st;
18        if (i == z || j == z || i + j == z) {
19            return true;
20        }
21        stk.emplace(x, j);
22        stk.emplace(i, y);
23        stk.emplace(0, j);
24        stk.emplace(i, 0);
25    }
26    return false;
27 }
```

分割回文子串

代码实现

```
class Solution {
public:
    vector<vector<string>> partition(string s) {
        int n = s.size();
        vector<vector<bool>> dp(n, vector<bool>(n, true));
        vector<vector<string>> res;
        vector<string> one_part;

        // 创建动态规划数组
        for(int i = n - 1; i >= 0; i--) {
            for(int j = i + 1; j < n; j++) {
                dp[i][j] = (s[i] == s[j]) && dp[i + 1][j - 1];
            }
        }
    }
};
```

```
    }

    // 利用回溯算法去搜索
    auto dfs = [&](auto &&dfs,int i){
        if(i == n){
            res.push_back(one_part);
            return;
        }
        else{
            for(int j = i;j<n;j++){
                if(dp[i][j]){
                    one_part.push_back(s.substr(i,j-i+1));
                    dfs(dfs,j+1);
                    one_part.pop_back();
                }
            }
        }
    };

    dfs(dfs,0);
    return res;
}
```

131. 分割回文串

中等

给你一个字符串 s，请你将 s 分割成一些子串，使每个子串都是回文串。返回 s 所有可能的分割方案。

示例 1:
输入: s = "aab"
输出: [["a","a","b"],["aa","b"]]

示例 2:
输入: s = "a"
输出: [["a"]]

提示:
1 ≤ s.length ≤ 16

已解答

通过 32 / 32 个通过的测试用例

爱门无赦了 提交于 2025.05.30 15:37

官方题解 写题解

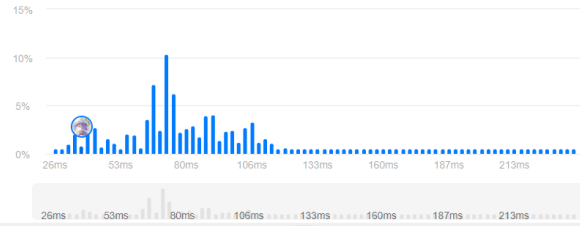
执行用时分布

38 ms | 击败 95.63%

复杂度分析

消耗内存分布

52.48 MB | 击败 87.05%



二十四点游戏

代码实现

```
class Solution {
public:
    static constexpr int TARGET = 24;
    static constexpr double EPSILON = 1e-6;
    static constexpr int ADD = 0, MULTIPLY = 1, SUBTRACT = 2, DIVIDE = 3;

    bool judgePoint24(vector<int> &nums) {
        vector<double> l;
        for (const int &num : nums) {
```

```

        l.emplace_back(static_cast<double>(num));
    }
    return solve(l);
}

bool solve(vector<double> &l) {
    if (l.size() == 0) {
        return false;
    }
    if (l.size() == 1) {
        return fabs(l[0] - TARGET) < EPSILON;
    }
    int size = l.size();
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            if (i != j) {
                vector<double> list2 = vector<double>();
                for (int k = 0; k < size; k++) {
                    if (k != i && k != j) {
                        list2.emplace_back(l[k]);
                    }
                }
                for (int k = 0; k < 4; k++) {
                    if (k < 2 && i > j) {
                        continue;
                    }
                    if (k == ADD) {
                        list2.emplace_back(l[i] + l[j]);
                    } else if (k == MULTIPLY) {
                        list2.emplace_back(l[i] * l[j]);
                    } else if (k == SUBTRACT) {
                        list2.emplace_back(l[i] - l[j]);
                    } else if (k == DIVIDE) {
                        if (fabs(l[j]) < EPSILON) {
                            continue;
                        }
                        list2.emplace_back(l[i] / l[j]);
                    }
                }
                if (solve(list2)) {
                    return true;
                }
                list2.pop_back();
            }
        }
    }
    return false;
}
};

```

题目描述

题解

提交记录

679. 24 点游戏

已解答

困难

相关标签

相关企业

Ax

给定一个长度为4的整数数组 `cards`。你有 4 张卡片，每张卡片上都包含一个范围在 `[1,9]` 的数字。您应该使用运算符 `{'+', '-', '*', '/'}` 和括号 `'('` 和 `')'` 将这些卡片上的数字排列成数学表达式，以获得值24。

你须遵守以下规则:

- 除法运算符 `'/'` 表示实数除法，而不是整数除法。
例如，`4 / (1 - 2 / 3)` = `4 / (1 / 3)` = `12`。
- 每个运算都在两个数字之间。特别是，不能使用 `"-"` 作为一元运算符。
例如，如果 `cards = [1,1,1,1]`，则表达式 `"-1 -1 -1 -1"` 是 **不允许** 的。
- 你不能把数字串在一起
例如，如果 `cards = [1,2,1,2]`，则表达式 `"12 + 12"` 无效。

如果可以得到这样的表达式，其计算结果为 `24`，则返回 `true`，否则返回 `false`。

代码

通过

全部提交记录

官方题解

写题解

通过

71 / 71 个通过的测试用例

爱门无赦了 提交于 2025.05.30 16:18

执行用时分布

消耗内存分布

12 ms | 击败 55.99%

11.32 MB | 击败 78.91%

复杂度分析